

Cheetah密钥封装算法

陶成东

2026 年 6 月 27 日

目录

1	前言	3
2	算法描述	5
2.1	数学基础	5
2.2	Cheetah公钥加密算法	14
2.3	Cheetah密钥封装算法	18
2.4	参数	19
3	设计原理	20
3.1	RLWE、MLWE与LWE问题	20
3.2	噪声分布	21
4	性能分析	21
4.1	性能	21
4.2	测试值	22
5	安全分析	23
5.1	安全证明	23
5.2	BKZ算法的复杂度	24
5.3	Cheetah密钥封装算法的实际安全评估	25
5.3.1	Primal Attack	25
5.3.2	Dual Attack	25
5.3.3	Cheetah密钥封装算法的攻击复杂度	26
6	优势和劣势	26

1 前言

Cheetah密钥封装算法是一族基于RLWE问题和MLWE问题的IND-CCA2安全的算法. 具体地, Cheetah128是基于RLWE问题的密钥封装算法, Cheetah256、Cheetah384和Cheetah512是基于MLWE问题的密钥封装算法. 我们首先构造CPA-安全的公钥加密算法, 然后再利用Fujisaki-Okamoto变换构造IND-CCA2安全的密钥封装算法.

为了便于理解, 我们在此简要地描述CPA-安全的Cheetah公钥加密算法的基本原理. 为了简便起见, 我们省略了公钥和密文的压缩和解压缩步骤, 以及NTT优化过程. 设 $N = 640$, $q = 7681$, k 为正整数, $\lambda < N$ 为安全参数. 设 $R_q = \mathbb{Z}_q[X]/X^N + 1$. Cheetah公钥加密算法的公钥和私钥生成步骤如下:

1. 随机生成多项式矩阵 $\mathbf{A} \in R_q^{k \times k}$.
2. 随机生噪声多项式向量 $\mathbf{s} \in R_q^k$ 和 $\mathbf{e} \in R_q^k$, 多项式的系数服从中心二项分布 ψ_η .
3. 计算 $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$.
4. 公钥为 $\mathbf{pk} = (\mathbf{A}, \mathbf{b})$, 私钥为 $\mathbf{sk} = \mathbf{s}$.

设 $\mathbf{m} \in \{0, 1\}^\lambda$ 是明文比特, 我们首先将明文表示成多项式 $\mathbf{m} = m_0 + m_1X + \dots + m_{\lambda-1}X^{\lambda-1} \in R_q$. 设 $\Delta = 2^{-1} \bmod q$, 则Cheetah公钥加密算法的加密步骤如下:

1. 随机生噪声多项式向量 $\mathbf{e}_1 \in R_q^k$, $\mathbf{e}_2 \in R_q^k$ 和 $\mathbf{e}_3 = e_{3,0} + e_{3,1}X + \dots + e_{3,\lambda-1}X^{\lambda-1} \in R_q$, 多项式的系数服从中心二项分布 ψ_η .
2. 计算 $\mathbf{c}_1^\top = \mathbf{e}_1^\top \mathbf{A} + \mathbf{e}_2^\top$.
3. 计算 $\mathbf{c}_2 = \text{Truncate}(\mathbf{e}_1^\top \mathbf{b}) + \mathbf{e}_3 + \Delta \cdot \mathbf{m}$, 其中 Truncate 函数参见2.1小节. 密文 $\mathbf{c}_2$ 只有 λ 个系数, 减少了密文的长度.
4. 密文为 $(\mathbf{c}_1, \mathbf{c}_2)$

Cheetah公钥加密算法的解密步骤如下:

1. 计算 $\mathbf{u} = 2(\mathbf{c}_2 - \text{Truncate}(\mathbf{c}_1^\top \mathbf{s})) \bmod \pm q$.
2. 将 $\mathbf{u}$ 看成整数多项式, 计算 $\mathbf{m} = \mathbf{u} \bmod 2$, 则 $\mathbf{m}$ 即是明文.

在以往的基于LWE问题的加密算法中, 解密算法的最后一步通常是比较密文与 $q/4$ 的大小, 如果大于 $q/4$, 则对应的明文比特为1, 否则对应的明文比特为0. 在Cheetah加密算法中, 我们只需要做模运算, 就可以恢复明文. 这与NTRU加密算法的解密过程相似, 提升了解密的速度.

我们将Cheetah公钥加密算法归纳总结为表1.

本文剩余部分结构如下: 第2节将详细描述Cheetah密钥封装算法和参数; 第3节介绍Cheetah密钥封装算法的设计原理; 第4节介绍Cheetah密钥封装算法的实现和性能; 第5节介绍Cheetah密钥封装算法的安全性; 第6节介绍Cheetah密钥封装算法的优势和劣势.

KeyGen:

- 1: $\mathbf{A} \leftarrow R_q^{k \times k}, \mathbf{s} \in R_q^k \leftarrow \psi_\eta, \mathbf{e} \in R_q^k \leftarrow \psi_\eta$
- 2: $\mathbf{b} \leftarrow \mathbf{A}\mathbf{s} + \mathbf{e}$
- 3: **return** $\mathbf{pk} = (\mathbf{A}, \mathbf{b}), \mathbf{sk} = \mathbf{s}$

Encrypt(pk, m):

- 1: $\mathbf{e}_1 \in R_q^k \leftarrow \psi_\eta, \mathbf{e}_2 \in R_q^k \leftarrow \psi_\eta, \mathbf{e}_3 \in R_q^k \leftarrow \psi_\eta$
- 2: $\mathbf{c}_1^\top = \mathbf{e}_1^\top \mathbf{A} + \mathbf{e}_2^\top$
- 3: $\mathbf{c}_2 = \text{Truncate}(\mathbf{e}_1^\top \mathbf{b}) + \mathbf{e}_3 + \Delta \cdot \mathbf{m}$
- 4: **return** $(\mathbf{c}_1, \mathbf{c}_2)$

Decrypt(sk, c₁, c₂):

- 1: $\mathbf{u} \leftarrow 2(\mathbf{c}_2 - \text{Truncate}(\mathbf{c}_1^\top \mathbf{s})) \bmod \pm q$
 - 2: $\mathbf{m} \leftarrow \mathbf{u} \bmod 2$
 - 3: **return** $\mathbf{m}$
-

表 1: Cheetah加密算法

2 算法描述

2.1 数学基础

整数模运算 设 $\mathbb{Z}$ 为整数环, q 为素数, $\mathbb{Z}_q$ 表示商环 $\mathbb{Z}/q\mathbb{Z}$. 设 a 为整数, 模运算 $r = a \bmod q$ 表示 $r \in \{0, 1, \dots, q-1\}$, 模运算 $r = a \bmod^{\pm} q$ 表示 $r \in \{-\frac{q-1}{2}, \dots, -1, 0, 1, \dots, \frac{q-1}{2}\}$. 记 $d_q = \lceil \log_2(q) \rceil$.

范数 设 $a \in \mathbb{Z}_q$, $\mathbf{a} = (a_0, a_1, \dots, a_{n-1}) \in \mathbb{Z}_q^n$, 我们定义

$$\|a\|_{\infty} = |a \bmod^{\pm} q|, \quad \|\mathbf{a}\|_{\infty} = \max_{0 \leq i \leq n-1} \|a_i\|_{\infty}.$$

$$\|\mathbf{a}\|_2 = (\|a_0\|_{\infty}^2 + \dots + \|a_{n-1}\|_{\infty}^2)^{1/2}.$$

压缩函数 设 d 为正偶数, $x \in \{0, 1, \dots, q-1\}$, 我们定义压缩函数如下:

$$\text{Compress}(x, d) = \left\lfloor \frac{x}{d} \right\rfloor.$$

记 $y = \text{Compress}(x, d)$, 我们定义解压函数如下:

$$\text{Decompress}(y, d) = \min \left(y \cdot d + \frac{d}{2}, q-1 \right).$$

当 d 为2的方幂时, 我们只需利用移位运算和加法运算就可以实现 $\mathbb{Z}_q$ 的元素的压缩和解压缩, 而不需要除法运算和模运算.

记 $\tilde{x} = \text{Decompress}(\text{Compress}(x, d), d)$, 对任意 $x \in \mathbb{Z}_q$, 容易证明 $|x - \tilde{x}| \leq \frac{d}{2}$. 事实上, 根据取整运算的定义, 存在整数 $r \in [0, d)$, 使得 $x = yd + r$. 我们接下来分两种情况证明. 若 $yd + \frac{d}{2} \leq q-1$, 则 $|x - \tilde{x}| = |r - \frac{d}{2}| \leq \frac{d}{2}$. 若 $yd + \frac{d}{2} > q-1$, 则 $\tilde{x} = q-1$. 另一方面 $x \geq yd > (q-1) - \frac{d}{2}$ (否则 $yd + \frac{d}{2} \leq q-1$). 所以 $|x - \tilde{x}| = (q-1) - x < \frac{d}{2}$.

设 $\mathbf{a} \in \mathbb{Z}_q^n$, $\mathbf{A} \in \mathbb{Z}_q^{n \times n}$, $\text{Compress}(\mathbf{a}, d)$ 和 $\text{Compress}(\mathbf{A}, d)$ 分别表示对向量 $\mathbf{a}$ 和矩阵 $\mathbf{A}$ 的每个元素做压缩运算. $\text{Decompress}(\mathbf{a}, d)$ 和 $\text{Decompress}(\mathbf{A}, d)$ 分别表示对向量 $\mathbf{a}$ 和矩阵 $\mathbf{A}$ 的每个元素做解压缩运算.

比特串与字节数组 设集合 $\mathbb{B} = \{0, 1, 2, \dots, 255\}$, t 为正整数. 算法1的功能是将比特数组 $\mathbf{b}$ 转换为字节数组 $\mathbf{bytes}$. 设比特串为 $b_0b_1b_2b_3b_4b_5b_6b_7$, 则对应的字节为 $b_0 + b_1 \cdot 2 + b_2 \cdot 2^2 + \dots + b_7 \cdot 2^7$, 即我们采用小端位序(LSB first order). 例如, 比特串11111011对应的字节为 $2^0 + 2^1 + 2^2 + 2^3 + 2^4 + 2^6 + 2^7 = 223$.

算法 1 BitsToBytes(**b**):比特数组转换为字节数组.

输入: 比特数组 $\mathbf{b} \in \{0, 1\}^{8t}$.

输出: 字节数组 $\mathbf{bytes} \in \mathbb{B}^t$.

```
1: bytes  $\leftarrow (0, \dots, 0)$ 
2: for ( $i = 0; i < t; i++$ ) do
3:    $\mathbf{bytes}[i] \leftarrow \sum_{j=0}^7 \mathbf{b}[8i + j] \cdot 2^j$ 
4: end for
5: return bytes
```

算法2的功能是将字节数组 $\mathbf{bytes}$ 转换为比特数组 $\mathbf{b}$. 设 $\mathbf{bytes}$ 的某个元素为 $b_0 + b_1 \cdot 2 + \dots + b_7 \cdot 2^7$, 则它对应的比特串为 $b_0 b_1 \dots b_7$.

算法 2 BytesToBits(**bytes**):字节数组转换为比特数组.

输入: 字节数组 $\mathbf{bytes} \in \mathbb{B}^t$.

输出: 比特数组 $\mathbf{b} \in \{0, 1\}^{8t}$.

```
1: c  $\leftarrow \mathbf{bytes}$ 
2: for ( $i = 0; i < t; i++$ ) do
3:   for ( $j = 0; j < 8; j++$ ) do
4:      $\mathbf{b}[8i + j] \leftarrow \mathbf{c}[i] \bmod 2$ 
5:      $\mathbf{c}[i] \leftarrow \lfloor \mathbf{c}[i] / 2 \rfloor$ 
6:   end for
7: end for
8: return b
```

整数数组与字节数组 设 d 为正整数, 算法3的功能是将整数数组转换为字节数组, 其中整数数组的元素定义在区间 $[0, 2^d - 1]$ 之中, 数组的长度为 n . 在Cheetah密钥封装算法中, n 是8的倍数. 设整数数组 $\mathbf{c}$ 的某个元素为 $b_0 + b_1 \cdot 2 + \dots + b_{d-1} \cdot 2^{d-1}$, 则它对应的比特串为 $b_0 b_1 \dots b_{d-1}$. 函数IntegersToBytes首先将长度为 n 的整数数组 $c_0, c_1, \dots, c_{n-1}$ 转换为比特串 $b_0, b_1, \dots, b_{nd-1}$, 然后再利用BitsToBytes函数把比特串转换为字节数组.

算法 3 $\text{IntegersToBytes}(\mathbf{c}, n, d)$: 整数数组转换为字节数组.

输入: 整数数组 $\mathbf{c} \in \mathbb{Z}_{2^d}^n$, 数组长度 n , 正整数 d .

输出: 字节数组 $\mathbf{bytes} \in \mathbb{B}^{nd/8}$.

```

1:  $\mathbf{b} \leftarrow (0, 0, \dots, 0)$   $\triangleright$   $\mathbf{b}$  的长度为  $nd$ .
2: for ( $i = 0; i < n; i++$ ) do
3:    $a \leftarrow \mathbf{c}_i$ 
4:   for ( $j = 0; j < d; j++$ ) do
5:      $\mathbf{b}[i \cdot d + j] \leftarrow a \bmod 2$ 
6:      $a \leftarrow (a - \mathbf{b}[i \cdot d + j]) / 2$ 
7:   end for
8: end for
    $\mathbf{bytes} \leftarrow \text{BitsToBytes}(\mathbf{b})$ 
9: return  $\mathbf{bytes}$ 

```

算法4的功能是将字节数组转换为整数数组. 首先利用 BytesToBits 函数将字节数组转换为比特串, 比特串的排序为小端位序. 然后再将比特串转换为整数数组.

算法 4 $\text{BytesToIntegers}(\mathbf{bytes}, n, d)$: 字节数组转换为整数数组.

输入: 字节数组 $\mathbf{bytes} \in \mathbb{B}^{nd/8}$, 正整数 n 和 d .

输出: 整数数组 $\mathbf{c} \in \mathbb{Z}_{2^d}^n$.

```

1:  $\mathbf{b} \leftarrow \text{BytesToBits}(\mathbf{bytes})$ 
2: for ( $i = 0; i < n; i++$ ) do
3:    $\mathbf{c}[i] \leftarrow \sum_{j=0}^{d-1} \mathbf{b}[i \cdot d + j] \cdot 2^j \bmod 2^d$ 
4: end for
5: return  $\mathbf{c}$ 

```

多项式环 设 $N = 640, q = 7681$, 我们定义多项式环 $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$. $\mathcal{R}$ 和 $\mathcal{R}_q$ 的多项式的系数分别定义在 $\mathbb{Z}$ 和 $\mathbb{Z}_q$ 上. 我们用粗体小写字母来表示 $\mathcal{R}_q$ 的元素, 对任意 $f(X) = f_0 + f_1X + \dots + f_{n-1}X^{N-1} \in \mathcal{R}_q$, 我们通常将它记为 $\mathbf{f} = (f_0, f_1, \dots, f_{n-1})$. 在不引起歧义的前提下, 粗体小写字母有时表示多项式, 有时表示多项式的系数向量. 设 η 为正整数, 且 $\eta < q$, $\mathbf{e} \in \mathcal{R}_q$, 如果 $\mathbf{e}$ 的所有系数都在集合 $\{-\eta, \dots, -1, 0, 1, \dots, \eta\}$ 之中,

我们称 $\mathbf{e}$ 为噪声多项式或者错误多项式. 设 $\mathbf{m} \in \mathcal{R}_q$, 如果 $\mathbf{m}$ 的所有系数都在集合 $\{0, 1\}$ 之中, 我们称 $\mathbf{m}$ 为二元多项式. 设 $\mathbf{v} = v_0 + v_1X + \cdots + v_{n-1}X^{N-1} \in R_q$, λ 为正整数, 且 $\lambda < N$, 我们定义 $\mathbf{v}$ 的截断多项式如下:

$$\text{Truncate}(\mathbf{v}, \lambda) = v_0 + v_1X + \cdots + v_{\lambda-1}X^{\lambda-1}.$$

Cheetah密钥封装算法的主要运算为 $\mathcal{R}_q$ 的多项式加法和乘法运算, 其中乘法运算可用NTT变换实现.

NTT变换 设 ψ 为 $\mathbb{Z}_q$ 的 $2N$ 阶本原单位根, $\omega = \psi^2$ 为 $\mathbb{Z}_q$ 的 N 阶本原单位根. 设 $\mathbf{a} = a_0 + a_1X + \cdots + a_{N-1}X^{N-1} \in R_q$, 多项式 $\mathbf{a}$ 的数论变换定义为

$$\text{NTT}(\mathbf{a}) = \hat{\mathbf{a}} = \hat{a}_0 + \hat{a}_1X + \cdots + \hat{a}_{N-1}X^{N-1},$$

其中 $\hat{a}_i = \sum_{j=0}^{n-1} \psi^{j(2i+1)} a_j \pmod{q}$ ($i = 0, 1, \dots, N-1$). 向量 $\hat{\mathbf{a}}$ 的逆数论变换定义为

$$\text{INTT}(\hat{\mathbf{a}}) = \mathbf{a} = a_0 + a_1X + \cdots + a_{N-1}X^{N-1},$$

其中 $a_i = N^{-1} \sum_{j=0}^{n-1} \psi^{-i(2j+1)} \hat{a}_j \pmod{q}$ ($i = 0, 1, \dots, N-1$). 在Cheetah密钥封装算法中, $q = 7681$, $N = 640 = 128 \times 5$. 记 $N_1 = 128$, $N_2 = 5$. 我们用Good-Thomas算法来快速计算 NTT_{640} 和 INTT_{640} . 具体计算过程如算法5所示. 算法5的输入为向量 $\mathbf{a}$ 和 dir . 当 $dir = \text{FORWARD}$ 时, 算法的输出为 $\mathbf{z} = \text{NTT}_{640}(\mathbf{a})$. 当 $dir = \text{INVERSE}$ 时, 算法的输出为 $\mathbf{z} = \text{INTT}_{640}(\mathbf{a})$. 算法的第6-10行将向量 $\mathbf{a}$ 转化为 N_1 行 N_2 列的二维数组, 第11-13行对二维数组的每一列做 NTT_{128} 变换, 第14-16行对二维数组的每一行做 NTT_5 变换, 第17-21行将二维数组转化为一维向量. NTT_5 变换如算法6所示, 我们可以通过硬编码来实现 NTT_5 变换. NTT_{128} 变换如算法7所示. 我们用Cooley-Tukey算法实现 NTT_{128} 变换. 算法7的`bit_reverse`函数定义如下: 设 z 为正整数, z 的二进制表示为 $z = z_d \cdot 2^d + \cdots + z_1 \cdot 2 + z_0$, 其中 $z_j \in \{0, 1\}$ ($j = 0, 1, \dots, d$), 则

$$\text{bit_reverse}(z) = z_0 \cdot 2^d + \cdots + z_{d-1} \cdot 2 + z_d.$$

算法 5 $\text{NTT}_{640}(\mathbf{a}, \text{dir})$

输入: $\mathbf{a} \in \mathbb{Z}_q^N$ 和 $\text{dir} \in \{\text{FORWARD}, \text{INVERSE}\}$.

输出: 变换结果 $\mathbf{z}$.

```
1: if  $\text{dir} == \text{FORWARD}$  then
2:   for  $i = 0$  to  $N - 1$  do
3:      $\mathbf{a}[i] \leftarrow \mathbf{a}[i] \cdot \psi^i \bmod q$ 
4:   end for
5: end if
6: for  $n_1 = 0$  to  $N_1 - 1$  do
7:   for  $n_2 = 0$  to  $N_2 - 1$  do
8:      $\mathbf{A}[n_1, n_2] \leftarrow \mathbf{a}[(n_1 \cdot N_2 + n_2 \cdot N_1) \bmod N]$ 
9:   end for
10: end for
11: for  $n_2 = 0$  to  $N_2 - 1$  do
12:    $\tilde{\mathbf{A}}[\cdot][n_2] \leftarrow \text{NTT}_{128}(\mathbf{A}[\cdot][n_2], \text{dir})$ 
13: end for
14: for  $n_1 = 0$  to  $N_1 - 1$  do
15:    $\hat{\mathbf{A}}[n_1][\cdot] \leftarrow \text{NTT}_5(\tilde{\mathbf{A}}[n_1][\cdot], \text{dir})$ 
16: end for
17: for  $n_1 = 0$  to  $N_1 - 1$  do
18:   for  $n_2 = 0$  to  $N_2 - 1$  do
19:      $\mathbf{z}[(n_1 \cdot N_2 + n_2 \cdot N_1) \bmod N] \leftarrow \hat{\mathbf{A}}[n_1, n_2]$ 
20:   end for
21: end for
22: if  $\text{dir} == \text{INVERSE}$  then
23:   for  $i = 0$  to  $N - 1$  do
24:      $\mathbf{z}[i] \leftarrow \mathbf{z}[i] \cdot N^{-1} \cdot \psi^{-i} \bmod q$ 
25:   end for
26: end if
27: return  $\mathbf{z}$ 
```

算法 6 NTT₅(**a**, **dir**)

输入: $\mathbf{a} = (a_0, a_1, a_2, a_3, a_4) \in \mathbb{Z}_q^5$ 和 $\text{dir} \in \{\text{FORWARD}, \text{INVERSE}\}$.

输出: $\mathbf{z} = (z_0, z_1, z_2, z_3, z_4)$.

- 1: If $\text{dir} == \text{FORWARD}$ then $\theta \leftarrow \omega_5$, else $\theta \leftarrow \omega_5^{-1}$ $\triangleright \omega_5^5 \bmod q = 1$.
 - 2: $z_0 \leftarrow (a_0 + a_1 + a_2 + a_3 + a_4) \bmod q$
 - 3: $z_1 \leftarrow (a_0 + a_1\theta + a_2\theta^2 + a_3\theta^3 + a_4\theta^4) \bmod q$
 - 4: $z_2 \leftarrow (a_0 + a_1\theta^2 + a_2\theta^4 + a_3\theta + a_4\theta^3) \bmod q$
 - 5: $z_3 \leftarrow (a_0 + a_1\theta^3 + a_2\theta + a_3\theta^4 + a_4\theta^2) \bmod q$
 - 6: $z_4 \leftarrow (a_0 + a_1\theta^4 + a_2\theta^3 + a_3\theta^2 + a_4\theta) \bmod q$
 - 7: **return** $\mathbf{z} = (z_0, z_1, z_2, z_3, z_4)$
-

算法 7 NTT₁₂₈(**a**, **dir**)

输入: $\mathbf{a} \in \mathbb{Z}_q^{128}$ 和 $\text{dir} \in \{\text{FORWARD}, \text{INVERSE}\}$.

输出: 变换结果 $\mathbf{z}$.

- 1: $\tilde{\mathbf{a}} \leftarrow \text{bit_reverse}(\mathbf{a})$
 - 2: **if** $\text{dir} = \text{FORWARD}$ **then**
 - 3: $\text{twiddle_table} \leftarrow \{\omega_{128}^0, \omega_{128}^1, \dots, \omega_{128}^{127}\}$ $\triangleright \omega_{128}^{128} \bmod q = 1$.
 - 4: **else**
 - 5: $\text{twiddle_table} \leftarrow \{\omega_{128}^0, \omega_{128}^{-1}, \dots, \omega_{128}^{-127}\}$
 - 6: **end if**
 - 7: **for** ($m \leftarrow 1; m \leq 7; m \leftarrow m + 1$) **do**
 - 8: $\text{step} \leftarrow 2^m; \text{half} \leftarrow 2^{m-1}; \text{stride} \leftarrow N_1/2^m;$
 - 9: **for** ($i \leftarrow 0; i < 128; i \leftarrow i + \text{step}$) **do**
 - 10: **for** ($j \leftarrow 0; j < \text{half}; j \leftarrow j + 1$) **do**
 - 11: $u \leftarrow \tilde{\mathbf{a}}[i + j + \text{half}] \cdot \text{twiddle_table}[\text{stride} \cdot j] \bmod q$
 - 12: $\tilde{\mathbf{a}}[i + j + \text{half}] \leftarrow \tilde{\mathbf{a}}[i + j] - u$ $\triangleright \text{Lazy mod } q$
 - 13: $\tilde{\mathbf{a}}[i + j] \leftarrow \tilde{\mathbf{a}}[i + j] + u$ $\triangleright \text{Lazy mod } q$
 - 14: **end for**
 - 15: **end for**
 - 16: **end for**
 - 17: $\mathbf{z}[i] \leftarrow \tilde{\mathbf{a}}[i] \bmod q, \quad i = 0, 1, \dots, 127$
 - 18: **return** $\mathbf{z}$
-

设 $\mathbf{a} \in R_q$, $\text{NTT}_{640}(\mathbf{a}, \text{FORWARD})$ 和 $\text{NTT}_{640}(\mathbf{a}, \text{INVERSE})$ 分别表示对 $\mathbf{a}$ 的系数向量做正向和逆向NTT变换. 设 $\mathbf{b} \in R_q^k$ 为多项式向量, $\text{NTT}_{640}(\mathbf{b}, \text{FORWARD})$ 和 $\text{NTT}_{640}(\mathbf{b}, \text{INVERSE})$ 分别表示对多项式向量 $\mathbf{b}$ 的每个多项式做正向和逆向NTT变换. 设 $\mathbf{A} \in R_q^{k \times k}$, $\text{NTT}_{640}(\mathbf{A}, \text{FORWARD})$ 和 $\text{NTT}_{640}(\mathbf{A}, \text{INVERSE})$ 分别表示对多项式矩阵 $\mathbf{A}$ 的每个多项式做正向和逆向NTT变换.

设 $\mathbf{a} \in R_q$, $\mathbf{b} \in R_q$, $\hat{\mathbf{a}} = \text{NTT}_{640}(\mathbf{a}, \text{FORWARD}) = (\hat{a}_0, \hat{a}_1, \dots, \hat{a}_{N-1})$, $\hat{\mathbf{b}} = \text{NTT}_{640}(\mathbf{b}, \text{FORWARD}) = (\hat{b}_0, \hat{b}_1, \dots, \hat{b}_{N-1})$, 我们记

$$\hat{\mathbf{a}} \circ \hat{\mathbf{b}} = \text{NTT}_{640}((\hat{a}_0 \hat{b}_0, \hat{a}_1 \hat{b}_1, \dots, \hat{a}_{N-1} \hat{b}_{N-1}), \text{INVERSE}).$$

设 $\mathbf{u} = (\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{k-1}) \in R_q^k$, $\mathbf{v} = (\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_{k-1}) \in R_q^k$, $\hat{\mathbf{u}} = \text{NTT}_{640}(\mathbf{u}, \text{FORWARD})$, $\hat{\mathbf{v}} = \text{NTT}_{640}(\mathbf{v}, \text{FORWARD})$, 我们记

$$\hat{\mathbf{u}} \circ \hat{\mathbf{v}} = \sum_{i=0}^{k-1} \hat{\mathbf{u}}_i \circ \hat{\mathbf{v}}_i.$$

设 $\mathbf{A} \in R_q^{k \times k}$, $\mathbf{s} \in R_q^k$, $\hat{\mathbf{A}} = \text{NTT}_{640}(\mathbf{A}, \text{FORWARD})$, $\hat{\mathbf{s}} = \text{NTT}_{640}(\mathbf{s}, \text{FORWARD})$, 我们记

$$\hat{\mathbf{A}} \circ \hat{\mathbf{s}} = (\hat{\mathbf{A}}_0 \circ \hat{\mathbf{s}}, \dots, \hat{\mathbf{A}}_{k-1} \circ \hat{\mathbf{s}}),$$

其中 $\hat{\mathbf{A}}_i (i = 0, 1, \dots, k-1)$ 为矩阵 $\hat{\mathbf{A}}$ 的第 i 行向量. 同理, 我们记

$$\hat{\mathbf{s}} \circ \hat{\mathbf{A}} = (\hat{\mathbf{s}} \circ \hat{\mathbf{A}}_0, \dots, \hat{\mathbf{s}} \circ \hat{\mathbf{A}}_{k-1}),$$

其中 $\hat{\mathbf{A}}_j (j = 0, 1, \dots, k-1)$ 为矩阵 $\hat{\mathbf{A}}$ 的第 j 列向量.

$\mathbb{Z}_q^t$ 的向量采样 算法8是向量空间 $\mathbb{Z}_q^t$ 的向量采样算法. 我们使用16位拒绝采样方法实现该算法. 算法输入长度为 $3t$ 的字节数组, 输出向量 $\mathbf{a} = (a_0, a_1, \dots, a_{t-1}) \in \mathbb{Z}_q^t$. 当 $q = 7681$, $\text{threshold} = 7681 \times 8 = 61448$, 单次拒绝概率为 $1 - 61448/2^{16} = 0.0623779296875$. 当输入为 $3t$ 个字节时, 最大的尝试次数为 $3t/2$, 采样失败的概率为

$$1 - \sum_{i=t}^{3t/2} \binom{3t/2}{i} \left(\frac{7681}{8192}\right)^i \left(\frac{511}{8192}\right)^{3t/2-i}.$$

在Cheetah密钥封装算法中, $t \geq N$, 因此采样失败的概率远小于 2^{-256} .

多项式采样 算法9是 $\mathcal{R}_q$ 的多项式采样算法. 算法的输入是 $3N$ 个字节长的数组, 输出是多项式 $\mathbf{f} \in \mathcal{R}_q$ 的系数向量 $(f_0, f_1, \dots, f_{N-1})$.

算法 8 `SampleVec(bytes, t)`: $\mathbb{Z}_q^t$ 的向量采样.

输入: 正整数 t 和字符串 `bytes` $\in \mathbb{B}^{3t}$.

输出: $(a_0, a_1, \dots, a_{t-1}) \in \mathbb{Z}_q^t$.

```

1:  $j \leftarrow 0$ 
2: for ( $i = 0; i < 3t$  and  $j < t; i = i + 2$ ) do
3:    $x \leftarrow \text{bytes}[i] + 2^8 \cdot \text{bytes}[i + 1]$ 
4:   if ( $x < \text{threshold}$ ) then
5:      $a_j \leftarrow x \bmod q$ 
6:      $j \leftarrow j + 1$ 
7:   end if
8: end for
9: return  $(a_0, a_1, \dots, a_{t-1})$ 

```

算法 9 `SamplePoly(bytes)`: 多项式采样.

输入: 字符串 `bytes` $\in \mathbb{B}^{3N}$.

输出: $\mathbf{f} = (f_0, f_1, \dots, f_{N-1})$.

```

1:  $\mathbf{f} \leftarrow \text{SampleVec}(\text{bytes}, N)$ 
2: return  $\mathbf{f}$ 

```

多项式矩阵采样 设多项式矩阵为

$$\mathbf{B} = \begin{pmatrix} \mathbf{b}_{00} & \mathbf{b}_{01} & \cdots & \mathbf{b}_{0,v-1} \\ \mathbf{b}_{10} & \mathbf{b}_{11} & \cdots & \mathbf{b}_{1,v-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{b}_{u-1,0} & \mathbf{b}_{u-1,1} & \cdots & \mathbf{b}_{u-1,v-1} \end{pmatrix},$$

其中 $\mathbf{b}_{ij} \in R_q$ ($0 \leq i < u, 0 \leq j \leq v-1$). 我们将多项式的系数排列成一维向量:

$$\mathbf{b} = (\mathbf{b}_{00}, \dots, \mathbf{b}_{0,v-1}, \mathbf{b}_{1,0}, \dots, \mathbf{b}_{1,v-1}, \dots, \mathbf{b}_{u-1,v-1}) \in \mathbb{Z}_q^{uvN}.$$

则多项式矩阵采样算法如下:

噪声向量采样 Cheetah 密钥封装算法的噪声多项式的系数根据中心二项分布来采样. 设 η 为正整数, 算法 11 是噪声向量采样算法. 算法的输入为 $2t\eta/8$ 个字节长的数组 $\mathbf{b}$, 输出噪声向量 $\mathbf{e} \in \mathbb{Z}_q^t$.

算法 10 $\text{SamplePolyMat}(\text{bytes}, u, v)$: 多项式矩阵采样.

输入: 正整数 u 和 v , 字符串 $\text{bytes} \in \mathbb{B}^{3uvN}$.

输出: $\mathbf{b} = (\mathbf{b}_{00}, \dots, \mathbf{b}_{0,v-1}, \mathbf{b}_{1,0}, \dots, \mathbf{b}_{1,v-1}, \dots, \mathbf{b}_{u-1,v-1}) \in \mathbb{Z}_q^{uvN}$.

1: $\mathbf{b} \leftarrow \text{SampleVec}(\text{bytes}, uvN)$

2: **return** $\mathbf{b}$

算法 11 $\text{SampleNoiseVec}(\mathbf{b}, t)$: 噪声多项式采样.

输入: 正整数 t 和字节数组 $\mathbf{b} \in \mathbb{B}^{2t\eta/8}$.

输出: 噪声向量 $\mathbf{e} \in \mathbb{Z}_q^t$.

1: $(b_0, b_1, \dots, b_{2t\eta-1}) \leftarrow \text{BytesToBits}(\mathbf{b})$

2: **for** $(i = 0; i < t; i++)$ **do**

3: $a \leftarrow \sum_{j=0}^{\eta-1} b_{2i\eta+j}$

4: $b \leftarrow \sum_{j=0}^{\eta-1} b_{2i\eta+\eta+j}$

5: $e_i \leftarrow a - b$

6: **end for**

7: **return** $\mathbf{e} = (e_0, e_1, \dots, e_{t-1})$.

噪声多项式向量采样 设 $\mathbf{e} = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_k) \in R^k$ 为噪声多项式向量, 这些向量的系数服从中心二项分布. 噪声多项式向量的采样算法如算法12所示. 算法的输入为字节数组 bytes , 输出噪声多项式向量的系数所组成的向量 $\mathbf{e}$.

算法 12 $\text{SampleNoisePolyVec}(\text{bytes}, k)$: 噪声多项式采样.

输入: 字符串 $\text{bytes} \in \mathbb{B}^{2kN\eta/8}$.

输出: $\mathbf{e} = (e_{00}, \dots, e_{0,N-1}, e_{1,0}, \dots, e_{1,N-1}, \dots, e_{k-1,N-1}) \in \mathbb{Z}_q^{kN}$.

1: $\mathbf{e} \leftarrow \text{SampleNoiseVec}(\text{bytes}, kN)$

2: **return** $\mathbf{e}$

多项式向量的编码和解码 设多项式向量 $\mathbf{b} = (\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_{k-1}) \in R^k$, 且多项式的系数在区间 $[0, 2^d - 1]$ 之中, 我们将多项式向量的系数看成向量空间 $\mathbb{Z}^{kN}$ 中的向量, 则 $\mathbf{b}$ 的编码算法如算法13所示.

算法 13 EncodePolyVec($\mathbf{b}, k, d$): 多项式向量编码.

输入: 多项式向量 $\mathbf{b} \in R_q^k$, 正整数 k 和 d .

输出: 字节数组 $\mathbf{bytes} \in \mathbb{B}^{kNd/8}$.

- 1: $\mathbf{bytes} \leftarrow \text{IntegersToBytes}(\mathbf{b}, kN, d)$
 - 2: **return** $\mathbf{bytes}$
-

相应的解码算法如算法14所示.

算法 14 DecodePolyVec($\mathbf{bytes}, k, d$): 多项式向量解码.

输入: 字节数组 $\mathbf{bytes} \in \mathbb{B}^{kNd/8}$, 正整数 k 和 d .

输出: $\mathbf{b} \in R^k$.

- 1: $(\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{k-1}) \leftarrow \text{BytesToIntegers}(\mathbf{bytes}, kN, d)$
 - 2: **return** $(\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{k-1})$
-

辅助密码函数 Cheetah密钥封装算法需要用到密码学安全的伪随机函数PRF、杂凑函数H, G 和可扩展输出函数XOF.

2.2 Cheetah公钥加密算法

本小节介绍CPA-安全的Cheetah公钥加密算法.

生成公钥和私钥 算法15是Cheetah公钥和私钥生成算法, 其中 λ 为安全参数. 例如, 当密码算法的安全强度为256位经典比特时, $\lambda = 256$. 算法的输出为公钥和私钥. $\mathbf{s}$ 和 $\mathbf{e}$ 是噪声多项式向量, 每一个噪声多项式的系数都服从中心二项分布. 我们假设算法的第4步的矩阵 $\hat{\mathbf{A}}$ 为经过NTT变换的矩阵, 而不是普通矩阵, 这样可以减少NTT变换的次数.

算法 15 Cheetah_PKE_KeyGen():Cheetah密钥生成算法

输入: NULL.

输出: 公钥pk和私钥sk.

```

1:  $seed \in \mathbb{B}^{\lambda/8+16} \leftarrow \text{RNG}()$ 
2:  $(\rho, \sigma, \gamma) \in \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16} \leftarrow \text{XOF}(seed)$ 
3:  $\mathbf{x} \leftarrow \text{XOF}(\rho)$   $\triangleright |\mathbf{x}| = 3k^2N.$ 
4:  $\hat{\mathbf{A}} \leftarrow \text{SamplePolyMat}(\mathbf{x}, k, k)$   $\triangleright \hat{\mathbf{A}} \in R_q^{k \times k}.$ 
5:  $\mathbf{y} \leftarrow \text{XOF}(\sigma)$   $\triangleright |\mathbf{y}| = 2kN\eta/8.$ 
6:  $\mathbf{s} \leftarrow \text{SampleNoisePolyVec}(\mathbf{y}, k)$   $\triangleright \mathbf{s} \in R_q^k.$ 
7:  $\mathbf{z} \leftarrow \text{XOF}(\gamma)$   $\triangleright |\mathbf{z}| = 2kN\eta/8.$ 
8:  $\mathbf{e} \leftarrow \text{SampleNoisePolyVec}(\mathbf{z}, k)$   $\triangleright \mathbf{e} \in R_q^k.$ 
9:  $\hat{\mathbf{s}} \leftarrow \text{NTT}_{640}(\mathbf{s}, \text{FORWARD})$ 
10:  $\mathbf{b} \leftarrow \text{Compress}(\hat{\mathbf{A}} \circ \hat{\mathbf{s}} + \mathbf{e}, 2^{\lceil \log_2(q) \rceil - d_b})$ 
11:  $\text{pk} \leftarrow \rho || \text{EncodePolyVec}(\mathbf{b}, k, d_b)$ 
12:  $\text{sk} \leftarrow \text{EncodePolyVec}(\hat{\mathbf{s}}, k, \lceil \log_2(q) \rceil)$ 
13: return (pk, sk)

```

加密 算法16是Cheetah加密算法. 我们首先将 λ 比特长的明文编码成二元多项式 $\mathbf{m}$, 然后再进行加密. 在第6步中, 我们隐藏了 $\tilde{\mathbf{v}}$ 的部分系数, 只将 $\tilde{\mathbf{v}}$ 的前 λ 个系数发送给解密者, 这样做既不影响算法的安全, 又减少了密文的大小. 从安全的角度看, 我们掩藏了多项式 $\tilde{\mathbf{v}}$ 的 $N - \lambda$ 个系数, 敌手能够获得的LWE样本数就减少了 $N - \lambda$ 个, 增加了敌手分析的难度.

算法 16 Cheetah_PKE.Encrypt(pk, m, γ): Cheetah加密算法

输入: 公钥pk、明文 $\mathbf{m} \in R_q$ 和种子字符串 $\gamma = (\gamma_1, \gamma_2, \gamma_3) \in \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16}$, 其中 $\mathbf{m}$ 是二元多项式.

输出: 密文 $(\mathbf{c}_1, \mathbf{c}_2)$.

```

1:  $\rho \leftarrow \text{pk}[0 : \lambda/8 + 16 - 1]$ 
2:  $\mathbf{x} \leftarrow \text{XOF}(\rho)$   $\triangleright |\mathbf{x}| = 3k^2N$ .
3:  $\hat{\mathbf{A}} \leftarrow \text{SamplePolyMat}(\mathbf{x}, k, k)$   $\triangleright \hat{\mathbf{A}} \in R_q^{k \times k}$ .
4:  $\tilde{\mathbf{b}} \leftarrow \text{DecodePolyVec}(\text{pk}[\lambda/8 + 16 : \lambda/8 + 16 + kNd_b/8 - 1], k, d_b)$ 
5:  $\mathbf{b} \leftarrow \text{Decompress}(\tilde{\mathbf{b}}, 2^{\lceil \log_2(q) \rceil - d_b})$ 
6:  $\mathbf{y} \leftarrow \text{XOF}(\gamma_1)$   $\triangleright |\mathbf{y}| = 2kN\eta/8$ .
7:  $\mathbf{r} \leftarrow \text{SampleNoisePolyVec}(\mathbf{y}, k)$   $\triangleright \mathbf{r} \in R_q^k$ .
8:  $\mathbf{z} \leftarrow \text{XOF}(\gamma_2)$   $\triangleright |\mathbf{z}| = 2kN\eta/8$ .
9:  $\mathbf{e}_1 \leftarrow \text{SampleNoisePolyVec}(\mathbf{z}, k)$   $\triangleright \mathbf{e}_1 \in R_q^k$ .
10:  $\mathbf{w} \leftarrow \text{XOF}(\gamma_3)$   $\triangleright |\mathbf{w}| = 2N\eta/8$ .
11:  $\mathbf{e}_2 \leftarrow \text{SampleNoisePolyVec}(\mathbf{w}, 1)$   $\triangleright \mathbf{e}_2 \in R_q$ .
12:  $\hat{\mathbf{r}} \leftarrow \text{NTT}_{640}(\mathbf{r}, \text{FORWARD})$   $\triangleright \hat{\mathbf{r}} \in R_q^k$ 
13:  $\mathbf{u} \leftarrow \text{Compress}(\hat{\mathbf{r}} \circ \hat{\mathbf{A}} + \mathbf{e}_1, 2^{\lceil \log_2(q) \rceil - d_u})$ 
14:  $\hat{\mathbf{b}} \leftarrow \text{NTT}_{640}(\mathbf{b}, \text{FORWARD})$ 
15:  $\tilde{\mathbf{v}} \leftarrow \text{Truncate}(\hat{\mathbf{r}} \circ \hat{\mathbf{b}} + \mathbf{e}_2, \lambda) + \Delta \cdot \mathbf{m}$ 
16:  $\mathbf{v} \leftarrow \text{Compress}(\tilde{\mathbf{v}}, 2^{\lceil \log_2(q) \rceil - d_v})$ 
17:  $\mathbf{c}_1 \leftarrow \text{EncodePolyVec}(\mathbf{u}, k, d_u)$ 
18:  $\mathbf{c}_2 \leftarrow \text{IntegersToBytes}(\mathbf{v}, \lambda, d_v)$ 
19: return  $(\mathbf{c}_1, \mathbf{c}_2)$ .

```

解密 算法17是Cheetah解密算法. 解密的过程与NTRU类似, 解密速度很快.

算法 17 Cheetah_PKE.Decrypt(sk, c₁, c₂): Cheetah解密算法

输入: 私钥sk和密文(c₁, c₂).

输出: 明文m.

```

1:  $\mathbf{u} \leftarrow \text{DecodePolyVec}(\mathbf{c}_1, k, d_u)$ 
2:  $\mathbf{v} \leftarrow \text{BytesToIntegers}(\mathbf{c}_2, \lambda, d_v)$ 
3:  $\hat{\mathbf{s}} \leftarrow \text{DecodePolyVec}(\text{sk}, k, \lceil \log_2(q) \rceil)$ 
4:  $\mathbf{u}' \leftarrow \text{Decompress}(\mathbf{u}, 2^{\lceil \log_2(q) \rceil - d_u})$ .
5:  $\mathbf{v}' \leftarrow \text{Decompress}(\mathbf{v}, 2^{\lceil \log_2(q) \rceil - d_v})$ .
6:  $\hat{\mathbf{u}} \leftarrow \text{NTT}_{640}(\mathbf{u}', \text{FORWARD})$ 
7:  $\mathbf{m}' \leftarrow (2 \cdot (\mathbf{v}' - \text{Truncate}(\hat{\mathbf{u}} \circ \hat{\mathbf{s}}, \lambda)) \bmod^{\pm} q) \bmod 2$ .
8:  $\mathbf{m} \leftarrow \text{IntegersToBytes}(\mathbf{m}', \lambda, 1)$ 
9: return m.

```

正确性 注意到

$$\begin{aligned}
& 2 \cdot (\mathbf{v}' - \text{Truncate}(\hat{\mathbf{u}} \circ \hat{\mathbf{s}}, \lambda)) \bmod^{\pm} q \\
&= 2 \cdot (\text{Decompress}(\mathbf{v}, 2^{d_q - d_v}) - \text{Truncate}(\mathbf{s} \cdot \mathbf{u}', \lambda)) \bmod^{\pm} q \\
&= 2 \cdot (\text{Decompress}(\text{Truncate}(\tilde{\mathbf{v}}, \lambda), 2^{d_q - d_v}) - \text{Truncate}(\mathbf{s} \cdot \mathbf{u}', \lambda)) \bmod^{\pm} q \\
&= 2 \cdot (\text{Truncate}(\text{Decompress}(\tilde{\mathbf{v}}, 2^{d_q - d_v}), \lambda) - \text{Truncate}(\mathbf{s} \cdot \text{Decompress}(\mathbf{u}, 2^{d_q - d_u}), \lambda)) \bmod^{\pm} q \\
&= \text{Truncate}(2 \cdot (\text{Decompress}(\tilde{\mathbf{v}}, 2^{d_q - d_v}) - \mathbf{s} \cdot \text{Decompress}(\mathbf{u}, 2^{d_q - d_u})), \lambda) \bmod^{\pm} q \\
&= \text{Truncate}(2 \cdot (\text{Decompress}(\text{Compress}(\mathbf{b} \cdot \mathbf{r} + \mathbf{e}_2 + c \cdot \mathbf{m}, 2^{d_q - d_v}), 2^{d_q - d_v}) \\
&\quad - \mathbf{s} \cdot \text{Decompress}(\text{Compress}(\mathbf{a} \cdot \mathbf{r} + \mathbf{e}_1, 2^{d_q - d_u}), 2^{d_q - d_u})), \lambda) \bmod^{\pm} q \\
&= \text{Truncate}(2 \cdot (\mathbf{b} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot (\mathbf{a} \cdot \mathbf{r} + \mathbf{e}_1 + \mathbf{e}'') + \mathbf{m}), \lambda) \bmod^{\pm} q \\
&= \text{Truncate}(2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}, \lambda) \bmod^{\pm} q,
\end{aligned}$$

其中, $\|\mathbf{e}'\|_{\infty} \leq 2^{d_q - d_v - 1}$, $\|\mathbf{e}''\|_{\infty} \leq 2^{d_q - d_u - 1}$. 如果 $\|\text{Truncate}(2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}, \ell)\|_{\infty} < q/2$, 那么

$$\begin{aligned}
& \text{Truncate}(2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}, \lambda) \bmod^{\pm} q \\
&= \text{Truncate}(2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}, \lambda),
\end{aligned}$$

我们就可以成功地解密密文. 又因为 $\|\text{Truncate}(2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}, \ell)\|_{\infty} \leq \|2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}\|_{\infty}$, 所以, 如果 $\|2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}\|_{\infty} < q/2$, 那么我们就可以成功地解密密文.

解密失败的概率 当 $\|2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}\|_\infty \geq q/2$ 时, 可能会解密失败. 我们接下来计算解密失败的概率. 一方面, 多项式 $\mathbf{s}, \mathbf{e}, \mathbf{r}, \mathbf{e}_1, \mathbf{e}_2$ 的系数服从中心二项分布 ψ_η . 另一方面, 我们选择适当的 d_u , 保证 $\mathbf{e}''$ 的系数在集合 $\{-\eta, \dots, -1, 0, 1, \dots, \eta\}$ 之中. 这样, 当 $\|\mathbf{e} \cdot \mathbf{r} + \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}''\|_\infty \geq q/4 - 2^{d_q-d_v-1} - \eta$ 时, 解密失败. 假设 $\mathbf{e}, \mathbf{r}, \mathbf{e}_1, \mathbf{s}, \mathbf{e}''$ 的系数服从均值为0标准差为 σ 的高斯分布, 则 $\mathbf{e} \cdot \mathbf{r} + \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}''$ 的系数近似服从均值为0标准差为 $\sqrt{3n}\sigma^2$ 的高斯分布, 于是 $\|\mathbf{e} \cdot \mathbf{r} + \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}''\|_\infty \geq q/4 - 2^{d_q-d_v-1} - \eta$ 的概率大约为

$$1 - \operatorname{erf}\left(\frac{q/4 - 2^{d_q-d_v-1} - \eta}{\sqrt{2}(\sqrt{3n}\sigma^2)}\right),$$

其中 $\operatorname{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2} dt$, $\sigma^2 = \eta/2$. 因此解密失败的概率大约为

$$n \cdot \left(1 - \operatorname{erf}\left(\frac{q/4 - 2^{d_q-d_v-1} - \eta}{\sqrt{2}(\sqrt{3n}\sigma^2)}\right)\right).$$

2.3 Cheetah密钥封装算法

我们用Fujisaki - Okamoto变换[1], 将Cheetah加密算法变成密钥封装算法. 算法18是密钥生成算法, 其中 $ek_{size} = \lambda/8 + kNd_b/8 + 16$, $dk_{size} = kN\lceil \log_2 q \rceil/8 + ek_{size} + |\mathbf{H}(ek)| + \lambda/8 + 16$.

算法 18 Cheetah_KEM_KeyGen()

输入: NULL

输出: $ek \in \mathbb{B}^{ek_{size}}$ 和 $dk \in \mathbb{B}^{dk_{size}}$

1: $salt \leftarrow \mathbb{B}^{\lambda/8+16}$

2: $(\mathbf{ek}_{\text{PKE}}, \mathbf{dk}_{\text{PKE}}) \leftarrow \text{Cheetah_PKE_KeyGen}()$

▷ 执行算法15.

3: $ek \leftarrow \mathbf{ek}_{\text{PKE}}$

4: $dk \leftarrow \mathbf{dk}_{\text{PKE}} \| ek \| \mathbf{H}(ek) \| salt$

5: **return** (ek, dk)

算法 19 Cheetah_KEM_Encaps($ek, \mathbf{m}$)

输入: 封装密钥 $ek \in \mathbb{B}^{ek_{size}}$ 和随机字符串 $\mathbf{m} \in \mathbb{B}^{\lambda/8}$.

输出: 共享密钥 $K \in \mathbb{B}^{\lambda/8}$ 和密文 $\mathbf{c} \in \mathbb{B}^{(kNd_u + \lambda d_v)/8}$.

- 1: $\mathbf{m} \leftarrow \mathbb{B}^{\lambda/8}$
 - 2: $(K, \gamma) \leftarrow \mathbf{G}(\mathbf{m} \parallel \mathbf{H}(ek))$
 - 3: $\mathbf{c} \leftarrow \text{Cheetah_PKE_Encrypt}(ek, \mathbf{m}, \gamma)$
 - 4: **return** $(K, \mathbf{c})$
-

算法 20 Cheetah_KEM_Decaps($dk, \mathbf{c}$)

输入: 解封密钥 $dk \in \mathbb{B}^{size_{dk}}$ 和密文 $\mathbf{c} \in \mathbb{B}^{(kNd_u + \lambda d_v)/8}$

输出: 共享密钥 $K \in \mathbb{B}^{\lambda/8}$

- 1: $(\mathbf{dk}_{\text{PKE}}, \mathbf{ek}_{\text{PKE}}, \mathbf{h}, \text{salt}) \leftarrow dk$
 - 2: $\mathbf{m}' \leftarrow \text{Cheetah_PKE_Decrypt}(\mathbf{dk}_{\text{PKE}}, \mathbf{c})$
 - 3: $(K', \gamma') \leftarrow \mathbf{G}(\mathbf{m}' \parallel \mathbf{h})$
 - 4: $\bar{K} \leftarrow \mathbf{J}(\text{salt} \parallel \mathbf{c})$
 - 5: $\mathbf{c}' \leftarrow \text{Cheetah_PKE_Encrypt}(\mathbf{ek}_{\text{PKE}}, \mathbf{m}', \gamma')$
 - 6: **if** $\mathbf{c} \neq \mathbf{c}'$ **then**
 - 7: $K' \leftarrow \bar{K}$
 - 8: **end if**
 - 9: **return** K'
-

2.4 参数

通过综合考虑安全、性能和解密失败的概率后, 我们选择Cheetah密钥封装算法的参数如表2所示.

	N	k	q	λ	η	(d_b, d_u, d_v)	δ
Cheetah128	640	1	7681	128	5	(10, 10, 4)	2^{-129}
Cheetah256	640	2	7681	256	4	(10, 10, 4)	2^{-176}
Cheetah384	640	3	7681	384	3	(11, 11, 4)	2^{-189}
Cheetah512	640	4	7681	512	2	(11, 11, 8)	2^{-243}

表 2: Cheetah密钥封装算法的参数

参数 λ 表示密钥封装算法的经典比特安全强度. Cheetah128、Cheetah256、Cheetah384和Cheetah512密钥封装算法对应的封装密钥长度和安全强度分别为: 128比特、256比特、384比特和512比特. 参数 $q = 7681$, 可以利用NTT变换快速计算多项式乘法. 参数 $N = 640$ 是模多项式的次数. k 是公钥多项式矩阵的维数. η 是中心二项分布的参数. d_b 是公钥压缩参数, d_u 和 d_v 是密文压缩参数. δ 是密钥解封装失败的概率.

参数选择说明 选择 $q = 7681$ 的原因如下: $7680 = 1280 \times 6$ 是一个NTT友好的素数.

选择 $N = 640$ 的原因如下: $N = 640 = 5 \times 2^7$, 当 $q = 7681$ 时, 我们可以用NTT变换来计算多项式的乘法. 根据标准的征集需求, 密钥封装算法需要支持512比特的安全, 因此, N 不能太小. 如果 N 太小, 例如, 当 $N < 512$ 时, 则需要多个多项式来负载512比特的明文信息, 或者用一个系数为多个比特的多项式来负载512比特信息. 从性能的角度考虑, 我们选择 $N = 640$.

参数 η, d_b, d_u, d_v 由解密失败的概率公式和安全分析公式决定.

3 设计原理

CheetahKEM密钥封装算法和Kyber的主要不同如下:

- 解密的逻辑不同. Kyber解密算法的最后一步是比较密文与 $q/4$ 的大小, 如果大于 $q/4$, 则对应的明文比特为1, 否则对应的明文比特为0. 在Cheetah加密算法中, 我们只需要做模运算, 就可以恢复明文. 这与NTRU加密算法的解密过程相似, 提升了解密的速度.
- 压缩和解压缩公钥和密文的方法不同, CheetahKEM只需利用移位运算和加法运算就可以实现 $\mathbb{Z}_q$ 的元素的压缩和解压缩, 而不需要除法运算和模运算.
- 参数选择不同. 为了满足512位经典比特安全, 我们选择 $N = 640$. 为了减少密文的长度, 我们用`Truncate`函数来截断密文多项式.

3.1 RLWE、MLWE与LWE问题

从安全的角度看, LWE问题没有代数结构, 而RLWE问题具有代数结构. RLWE和MLWE问题是LWE问题的一个特例. 如果存在某个算法能解

决LWE问题, 那么在相同的参数、时间复杂度和空间复杂度下, 这个算法也能解决RLWE和MLWE问题. 从性能的角度看, 基于RLWE 和MLWE问题的密码算法比基于LWE问题的密码算法更好. 基于RLWE和MLWE问题的密码算法的密钥长度、密文长度比基于LWE问题的密码算法更短, 而且加密和解密速度更快. 因此, 经过综合考虑密码算法的性能和安全, 我们利用RLWE和MLWE问题来设计CheetahKEM 密钥封装算法.

3.2 噪声分布

在基于LWE问题的密码系统中, 噪声分布通常有以下几种:

- 离散高斯分布, 例如FrodoKEM[2]等.
- 中心二项分布, 例如Kyber[3] 和Dilithium[4]等.
- NTRU类型分布, 即多项式的系数属于 $\{-1, 0, 1\}$, 且固定 -1 、 0 和 1 的数量. 例如NTRU加密算法[5]等.
- 均匀分布, 例如Saber[6]等.

由于中心二项分布能够在常数时间内快速实现, 可以更好地抵御侧信道攻击, 因此我们用中心二项分布来采样噪声多项式和噪声矩阵.

4 性能分析

4.1 性能

在存储公钥和私钥时, 我们的参考实现只存储生成公钥矩阵 $\hat{\mathbf{A}}$ 的种子比特, 其长度为 $\lambda/8 + 16$ 个字节. 在带宽和存储空间不受限制的情况下, 我们可以存储整个公钥矩阵 $\hat{\mathbf{A}}$ 的元素, 以加快密钥封装和解封装的速度.

Cheetah密钥封装算法的参考实现和AVX2实现的性能如表3所示.

Cheetah128		
size (Bytes)	ref (10 ³ Cycles)	avx2 (10 ³ Cycles)
sk : 1936	gen : 119	gen : 111
pk : 832	enc : 152	enc : 146
ct : 864	dec : 171	dec : 157
Cheetah256		
size (Bytes)	ref (10 ³ Cycles)	avx2 (10 ³ Cycles)
sk : 3808	gen : 308	gen : 289
pk : 1648	enc : 351	enc : 330
ct : 1728	dec : 388	dec : 356
Cheetah384		
size (Bytes)	ref (10 ³ Cycles)	avx2 (10 ³ Cycles)
sk : 5920	gen : 970	gen : 936
pk : 2704	enc : 1030	enc : 998
ct : 2832	dec : 1116	dec : 1068
Cheetah512		
size (Bytes)	ref (10 ³ Cycles)	avx2 (10 ³ Cycles)
sk : 7872	gen : 1568	gen : 1508
pk : 3600	enc : 1630	enc : 1578
ct : 4032	dec : 1753	dec : 1669

表 3: Cheetah密钥封装算法的性能

参考实现和AVX2实现的测试环境: CPU型号为Intel Core i7-1065G7, CPU的时钟频率为1.30GHz; 性能测试机器的内存大小为8G; 操作系统为Ubuntu 24.04.3 LTS, Linux内核版本为6.14.0; 编译器版本为gcc 13.3.0; 编译选项为-03 -march=native -fomit-frame-pointer; 表格中的Cycles数量为执行相关函数10000次后得到的平均值.我们的参考实现和AVX2实现都没有使用堆内存.

4.2 测试值

利用商用密码标准研究院提供的辅助程序和数据, 我们生成了所有算法实例的测试向量, 这些测试向量放在提交文件包的Test_Vector目录里, 具体

的测试向量文件为:

```
/ Test_Vector/KAT_KEM_Cheetah128.txt
/ Test_Vector/KAT_KEM_Cheetah256.txt
/ Test_Vector/KAT_KEM_Cheetah384.txt
/ Test_Vector/KAT_KEM_Cheetah512.txt
```

5 安全分析

5.1 安全证明

假设XOF, H和G是随机预言机, 判定性MLWE问题是困难的, 公钥 $\mathbf{A}$, $\mathbf{b}$ 没有被压缩, 我们首先证明Cheetah-PKE是IND-CPA安全的.

定理1. 假设判定性MLWE问题是困难的, 则Cheetah-PKE公钥加密算法在选择明文攻击下是IND-CPA安全的.

证明. 假设存在一个概率多项式时间(PPT)敌手 $\mathcal{A}$, 能以不可忽略的优势 ϵ 赢得IND-CPA游戏. 我们构造一个模拟器 $\mathcal{B}$ 来解决判定性MLWE问题. 假设 $\mathcal{B}$ 收到一个判定性MLWE挑战样本 $\mathbf{A}, \mathbf{b}$, 其中 $\mathbf{b}$ 要么是MLWE分布, 即 $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$, 要么是均匀随机矩阵. $\mathcal{B}$ 将挑战中的 $\mathbf{A}, \mathbf{b}$ 直接作为Cheetah-PKE的公钥发送给 $\mathcal{A}$. 当 $\mathcal{A}$ 提交两个明文 $\mathbf{m}_0, \mathbf{m}_1$ 时, $\mathcal{B}$ 随机选择比特 $\beta \in \{0, 1\}$. $\mathcal{B}$ 再次利用MLWE挑战的数学结构来生成密文 $(\mathbf{u}, \mathbf{v})$. 若 $\mathbf{b}$ 是真实MLWE样本, 则密文 $(\mathbf{u}, \mathbf{v})$ 的分布与真实Cheetah-PKE加密 $\mathbf{m}_\beta$ 的分布完全一致. 若 $\mathbf{b}$ 是均匀随机的, 则密文在统计上独立于明文 $\mathbf{m}_\beta$. 因此, $\mathcal{A}$ 猜对 β 的优势直接转化为 $\mathcal{B}$ 区分判定性MLWE样本的优势, 即

$$Adv_{IND-CPA}^{Cheetah-PKE}(\mathcal{A}) \leq 2 \cdot Adv_{MLWE}(\mathcal{B}).$$

□

利用Fujisaki-Okamoto变换, 我们把IND-CPA安全的Cheetah-PKE加密算法变成CheetahKEM 密钥封装算法. 接下来我们证明CheetahKEM在经典随机预言机模型下是紧的IND-CCA安全的, 在量子随机预言机模型下是非紧的IND-CCA安全的. 根据文献[7], 我们有

定理2 (Theorem 3.1 and 3.2 in [7]). 对任意敌手 $\mathcal{B}$, 假设 $\mathcal{B}$ 至多只能发起 q_D 次解密失败查询, q_G 次随机预言机 G 查询, q_H 次随机预言机 H 查询, 存在敌手 $\mathcal{A}$,

使得

$$Adv_{CheetahKEM}^{IND-CCA}(\mathcal{B}) \leq q_H \cdot \delta + (q_H + 2q_G + 1)/|\mathcal{M}| + 3 \cdot Adv_{Cheetah-PKE}^{IND-CPA}(\mathcal{A}),$$

其中 $\mathcal{A}$ 与 $\mathcal{B}$ 所需时间相同.

定理3 (Theorem 4.4 and 4.6 in [7]). 对任意量子敌手 $\mathcal{B}$, 假设 $\mathcal{B}$ 至多只能发起 q_D 次经典的解密失败查询, q_G 次量子随机预言机 $\mathcal{G}$ 查询, q_H 次量子随机预言机 $\mathcal{H}$ 查询, $q_{\hat{H}}$ 次量子随机预言机 $\hat{\mathcal{H}}$ 查询, 则存在量子敌手 $\mathcal{A}$, 使得

$$Adv_{CheetahKEM}^{IND-CCA}(\mathcal{B}) \leq (2q_{\hat{H}} + q_H) \cdot \sqrt{8 \cdot \delta(q_G + 1)^2 + (1 + 2q_G) \sqrt{Adv_{CheetahPKE}^{IND-CPA}(\mathcal{A})}},$$

其中 $\mathcal{A}$ 与 $\mathcal{B}$ 所需时间相同.

5.2 BKZ算法的复杂度

在对Cheetah密钥封装算法的安全性做评估之前, 我们首先介绍BKZ格约化算法的计算复杂度. BKZ算法是目前最好的寻找格的短向量的算法. 而在BKZ算法中, 又会用到枚举算法或者筛算法作为子算法. 在实际应用中, 用筛算法作为子算法的效果最好. 因此, 在本文中, 我们假设BKZ算法使用的子算法为筛算法. 设应用筛算法的子格的维数为 β , 根据文献[8, 9, 10, 11], 在经典计算机中, 筛算法的时间复杂度为 $2^{0.292\beta + o(\beta)}$, 空间复杂度为 $2^{0.2075\beta + o(\beta)}$. 根据文献[12, 13, 14, 15], 在量子计算机中, 筛算法的时间复杂度为 $2^{0.257\beta + o(\beta)}$, 同时需要相同数量级的量子空间复杂度. 由于BKZ算法的主要消耗在筛算法中, 因此, 我们将筛算法的计算复杂度作为BKZ算法的计算复杂度.

设格 $\mathcal{L}$ 的维数为 d , 接下来我们确定 β 的大小. 根据文献[16, 17], BKZ算法成功输出目标短向量 $\mathbf{s}$ 的条件为

$$\sqrt{\beta/d} \cdot \|\mathbf{s}\| < \delta_\beta^{2\beta-d-1} \cdot \mathbf{Vol}(\mathcal{L})^{1/d}, \quad (1)$$

其中 δ_β 为BKZ算法的root-Hermite-Factor, 根据文献[18], 当 $\beta > 50$ 时,

$$\delta_\beta = \left(\frac{(\pi\beta)^{1/\beta} \beta}{2\pi e} \right)^{1/(2\beta-2)}.$$

从而我们可以求出满足不等式(1)的最小的 β ($\beta > 50$), 进而评估BKZ算法的计算复杂度.

5.3 Cheetah密钥封装算法的实际安全评估

Cheetah密钥封装算法具有一些代数结构, 目前我们不知道如何有效地利用这些代数结构来攻击密码系统, 因此我们暂时忽略这些代数结构. 由于RLWE和MLWE问题可以看成为原始的LWE问题的一个变种, 因此我们可以用求解LWE问题的算法来求解RLWE和MLWE问题.

设Cheetah密钥封装算法的参数为 (q, k, N) , 公钥为 $(\mathbf{A}, \mathbf{b})$, 其中 $\mathbf{A} \in \mathbb{Z}^{m \times m}$, $\mathbf{b} \in \mathbb{Z}^m$, $m = kN$.

5.3.1 Primal Attack

构造格

$$\mathcal{L}_q(\mathbf{A}, \mathbf{I}_m, -\mathbf{b}) = \{\mathbf{x} \in \mathbb{Z}^{2m+1} : (\mathbf{A}, \mathbf{I}_m, -\mathbf{b})\mathbf{x} \bmod q = \mathbf{0}\}.$$

格的维数为 $d = 2m + 1$, 体积为 q^m . 容易验证 $(\mathbf{s}, \mathbf{e}, 1)$ 为格短向量, 于是我们可用BKZ算法来寻找这个短向量. BKZ算法成功找到这个短向量的条件为 $\sigma\sqrt{\beta} < \delta_\beta^{2\beta-d-1} \cdot q^{m/d}$, 其中 σ 为噪声向量的标准差.

在实际攻击过程中, 因为向量 $\mathbf{s}$ 和 $\mathbf{e}$ 的元素比较小, 所以我们可以猜测部分 $\mathbf{s}$ 或者 $\mathbf{e}$ 的元素的值, 然后再应用BKZ算法, 这样可以稍微减少格的维数, 从而降低BKZ算法的复杂度. 但是, 我们不能猜测太多 $\mathbf{s}$ 或者 $\mathbf{e}$ 的元素的值, 否则猜测的复杂度会很高. 我们需要平衡猜测的元素个数和BKZ算法的复杂度. 这就是所谓的Hybrid Primal Attack.

5.3.2 Dual Attack

我们首先构造对偶格

$$\mathcal{L}_q(\mathbf{A}) = \{(\mathbf{v}, \mathbf{w}) \in \mathbb{Z}^{2m} : \mathbf{w}^\top = \mathbf{v}^\top \mathbf{A} \bmod q\},$$

格的维数为 $d = 2m$, 体积为 q^m . 利用BKZ算法, 可以找到长度约为 $\ell = \delta_\beta^{d-1} q^{1/2}$ 的短向量 $(\mathbf{v}, \mathbf{w})$. 我们利用这个短向量来构造区分器, 以区分LWE样本和随机样本. 给定向量 $\mathbf{b} \in \mathbb{Z}^m$, 计算 $z \equiv \mathbf{v}^\top \mathbf{b} \equiv \mathbf{v}^\top \mathbf{A} \mathbf{s} + \mathbf{v}^\top \mathbf{e} \equiv \mathbf{w}^\top \mathbf{s} + \mathbf{v}^\top \mathbf{e} \pmod{q}$. 如果 $\mathbf{b} = \mathbf{A} \mathbf{s} + \mathbf{e}$, 那么 z 服从标准差为 $\ell\sigma$ 的高斯分布. 如果 $\mathbf{b}$ 服从均匀分布, 那么 z 也服从均匀分布. 高斯分布和均匀分布的统计距离为 $\epsilon = 4 \exp(-2\pi^2 \tau^2)$, 其中 $\tau = \ell\sigma/q$. 所以, 我们需要用BKZ算法生成 $1/\epsilon^2$ 个短向量. BKZ的筛算法生成 $2^{0.2075\beta}$ 个短向量, 假设这些短向量的长

度都不超过 ℓ , 我们需要执行 ϵ -区分器 $R = \max\{1, 1/2^{0.2075\beta\epsilon^2}\}$ 次, 才能最终恢复密钥(或者等价密钥) $\mathbf{s}$.

与Primal攻击相似, 在实际攻击过程中, 因为向量 $\mathbf{s}$ 和 $\mathbf{e}$ 的元素比较小, 所以我们可以猜测部分 $\mathbf{s}$ 或者 $\mathbf{e}$ 的元素的值, 然后再应用BKZ算法, 以降低攻击的复杂度.

5.3.3 Cheetah密钥封装算法的攻击复杂度

表4给出了Primal攻击和dual攻击Cheetah密钥封装算法的计算复杂度.

Algorithm	Primal Attack		Dual Attack	
	Classical	Quantum	Classical	Quantum
Cheetah128	159	139	157	137
Cheetah256	307	276	302	269
Cheetah384	426	382	418	375
Cheetah512	541	497	522	482

表 4: Primal攻击和dual攻击Cheetah密钥封装算法的计算复杂度, 表中的数据为攻击复杂度的以2为底的对数值.

6 优势和劣势

Cheetah密钥封装算法易于实现和扩展. 不同的安全强度的Cheetah密钥封装算法的实现只有头文件的宏定义不同, 其他代码可以复用. 如果需要调整安全强度, 只需要更改头文件的宏定义.

下面我们通过与其他后量子密钥封装方案的比较来说明Cheetah密钥封装算法的优势和劣势.

与直接基于LWE问题的方案相比 直接用LWE问题来构造密钥封装算法或密钥交换协议的方法主要有两种. 第一种方法详见文献[24], 这种方法的公钥和私钥都很大, 密文相对小. 另一种相对实用的方法详见文献[2], 即FrodoKEM密钥封装算法, 从表5 可见, FrodoKEM-640的公钥和密文长度大约为Cheetah密钥封装算法的11倍. 在安全方面, 直接基于LWE问题的密钥封装算法没有代数结构, 而Cheetah密钥封装算法有代数结构, 容易遭受潜在的代数攻击.

Scheme	sk	pk	ct	gen	enc	dec
Cheetah128	1936	832	864	119	152	171
Kyber512[19]	1632	800	768	123	155	188
FrodoKEM-640[2]	19888	9616	9720	1385	1862	1747
NewHope512[20]	1888	928	1120	117	180	206
ntruhs2048509[21]	935	699	699	8906	643	1662
hqc-128[22]	40	2249	4497	180	408	875
mceliece348864[23]	6492	261120	128	58034	44	135

表 5: 128比特安全的密钥封装算法比较, 其中**sk**、**pk**和**ct**的大小的单位为Bytes; **gen**、**enc** 和**dec**的单位为 10^3 Cycles; 它们均为参考实现的性能.

与基于MLWE问题的方案相比 典型的基于MLWE问题的密钥封装算法为Kyber密钥封装算法[19], 它是美国的后量子密钥封装标准算法. 在安全方面, Kyber512引入的代数结构为 $\mathbb{Z}_q[X]/X^{256} + 1$, 而Cheetah引入的代数结构为 $\mathbb{Z}_q[X]/X^{640} + 1$. 在性能方面, 从表5可见, Cheetah128的公钥和密文长度比Kyber512略长, Cheetah128密钥封装算法的公钥生成、密钥封装和密钥解封装速度比Kyber512密钥封装算法快.

与基于RLWE问题的方案相比 典型的基于RLWE问题的密钥封装算法为NewHope密钥封装算法[20]. 在安全方面, NewHope512密钥封装算法引入的代数结构为 $\mathbb{Z}_q[X]/X^{512} + 1$, 而Cheetah引入的代数结构为 $\mathbb{Z}_q[X]/X^{640} + 1$. 在性能方面, 从表5可见, Cheetah128密钥封装算法的公钥和密文长度比NewHope512密钥封装算法短, Cheetah128密钥封装算法的密钥封装和密钥解封装速度比NewHope512密钥封装算法快.

与NTRU相比 在文献[21]中, ntruhs2048509密钥封装方案的代数结构为 $\mathbb{Z}_q[X]/X^{509} - 1$, $q = 2048$. 在性能方面, 从表5可见, Cheetah128密钥封装算法的公钥和密文长度比ntruhs2048509密钥封装算法略长. Cheetah128密钥封装算法的公钥生成、密钥封装和密钥解封装速度比ntruhs2048509密钥封装算法快.

与基于编码问题的方案相比 典型的基于编码问题的密钥封装算法为Classic McEliece[23]密钥封装算法和HQC[22]密钥封装算法. 从表5可

见, 此两个算法的公钥和密文加起来的总长度比Cheetah128长, 公钥和私钥生成、密钥封装和解封装速度比Cheetah128慢.

参考文献

- [1] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Annual international cryptology conference*, pages 537–554. Springer, 1999.
- [2] Lewis Glabush, Patrick Longa, Michael Naehrig, Chris Peikert, Douglas Stebila, and Fernando Virdia. Frodokem: A cca-secure learning with errors key encapsulation mechanism. *Cryptology ePrint Archive*, 2025.
- [3] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European symposium on security and privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
- [4] Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, Marc Manzano, Ramesh Karri, Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, et al. Crystals-dilithium. *Hardware Architectures for Post-Quantum Digital Signature Schemes*, pages 13–30, 2021.
- [5] Jeffrey Hoffstein, Jill Pipher, and Joseph H Silverman. Ntru: A ring-based public key cryptosystem. In *International algorithmic number theory symposium*, pages 267–288. Springer, 1998.
- [6] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure kem. In *International Conference on Cryptology in Africa*, pages 282–305. Springer, 2018.
- [7] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography*, pages 341–371, Cham, 2017. Springer International Publishing.

- [8] Thijs Laarhoven. Sieving for shortest vectors in lattices using angular locality-sensitive hashing. In *Annual Cryptology Conference*, pages 3–22. Springer, 2015.
- [9] Anja Becker and Thijs Laarhoven. Efficient (ideal) lattice sieving using cross-polytope lsh. In *International Conference on Cryptology in Africa*, pages 3–23. Springer, 2016.
- [10] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In *Proceedings of the twenty-seventh annual ACM-SIAM symposium on Discrete algorithms*, pages 10–24. SIAM, 2016.
- [11] Martin R Albrecht, Léo Ducas, Gottfried Herold, Elena Kirshanova, Eamonn W Postlethwaite, and Marc Stevens. The general sieve kernel and new records in lattice reduction. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 717–746. Springer, 2019.
- [12] Thijs Laarhoven, Michele Mosca, and Van De Pol Joop. Finding shortest lattice vectors faster using quantum search. *Designs, Codes and Cryptography*, 77(2):375–400, 2015.
- [13] André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 63–91, 2021.
- [14] Max Heiser. Improved quantum hypercone locality sensitive filtering in lattice sieving. *Cryptology ePrint Archive*, 2021.
- [15] Divesh Aggarwal, Jianwei Li, Phong Q Nguyen, and Noah Stephens-Davidowitz. Slide reduction, revisited-filling the gaps in svp approximation. In *Annual International Cryptology Conference*, pages 274–295. Springer, 2020.
- [16] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange-a new hope. *Cryptology ePrint Archive*, 2015.

- [17] Tsuyoshi Takagi and Thomas Peyrin. Revisiting the expected cost of solving usvp and applications to lwe. *Springer, Cham*, 10.1007/978-3-319-70694-8(Chapter 11):297–322, 2017.
- [18] Yuanmi Chen and Phong Q. Nguyen. Bkz 2.0: Better lattice security estimates. *DBLP*, 2011.
- [19] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber algorithm specifications and supporting documentation. 2021.
- [20] Erdem Alkim, Roberto Avanzi, Joppe Bos, Léo Ducas, Antonio De La Piedra, Peter Schwabe Thomas Pöppelmann, and Douglas Stebila. Newhope. *Submission to round*, 1, 2017.
- [21] Cong Chen, Oussama Danba, Jeffrey Hoffstein, Andreas Hülsing, Joost Rijneveld, John M Schanck, Peter Schwabe, William Whyte, and Zhenfei Zhang. Algorithm specifications and supporting documentation. *Brown University and Onboard security company, Wilmington USA*, 2019.
- [22] Philippe Gaborit, Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, et al. Hamming quasi-cyclic (hqc). *NIST Post-Quantum Standardization, 4th Round; National Institute of Standards and Technology: Gaithersburg, MD, USA*, 2025.
- [23] Daniel J Bernstein, Tung Chou, Tanja Lange, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, et al. Classic mceliece: conservative code-based cryptography. *NIST submissions*, 1(1):1–25, 2017.
- [24] Chris Peikert, Vinod Vaikuntanathan, and Brent Waters. A framework for efficient and composable oblivious transfer. In *Annual international cryptology conference*, pages 554–571. Springer, 2008.